# Clothing Exchange & Swap Marketplace

Master Build Prompt & Project Brief

## Problem & Vision

Fast fashion has significantly increased clothing consumption, leading to textile waste and environmental issues. Many people have wearable clothes in good condition that they no longer use but find it inconvenient to sell or donate. This platform enables users to directly swap clothes with other users instead of purchasing new ones, supported by value-based swap suggestions and location-based matching — promoting a barter economy and sustainable fashion.

### Primary Objectives

- Provide a marketplace dedicated to clothing exchange
- Enable users to swap clothes directly without monetary transactions
- Encourage sustainable fashion practices
- Provide location-based swap matching

### In Scope / Out of Scope (Phase 1)

| In Scope | Out of Scope |
|---|---|
| User registration & login | Online payment system |
| Clothing listing system | AI-powered fashion recommendations |
| Swap request system | AR virtual clothing try-on |
| Negotiation chat, swap value calculator | Mobile application version |
| Location-based swap suggestions, admin panel | |

### Tech Stack

| Layer | Technology |
|---|---|
| Frontend | React.js, Bootstrap / Tailwind CSS |
| Backend | Node.js with Express.js |
| Database | MongoDB / PostgreSQL |
| Real-time | Socket.io (negotiation chat) |
| Deployment | AWS / Render / Vercel |

## Reference Documents

This brief is designed to be used alongside four companion project files — keep all five together in your repo's **/docs** folder:

- **rules.md** — what to use / avoid, libraries, error handling, AI boundaries, general standards
- **architecture.md** — system design, folder/file structure, tech stack, data model
- **phases.doc.md** — the 6-phase build breakdown (Auth → Dashboard → CRUD → Features → Testing → Deployment)
- **design.md** — UI/UX rules, color & theme system, typography, persisted UI preferences
- **memory.md** — running build log to preserve context across sessions

# Master Build Prompt

Paste the block below as your first message to an AI coding assistant (Claude Code, Cursor, etc.) once *rules.md*, *architecture.md*, *phases.doc.md*, *design.md*, and *memory.md* are in your repo's /docs folder.

You are acting as the lead full-stack engineer building the **Clothing Exchange & Swap Marketplace**, a platform that lets users swap wearable clothes directly with each other instead of buying new — reducing textile waste and promoting sustainable fashion, with value-based swap suggestions and location-based matching.

**Before writing any code**, read these five files in /docs and treat them as binding project context for the entire build: rules.md, architecture.md, phases.doc.md, design.md, and memory.md.

**Product summary:** users register, create a profile, upload clothing listings (type, size, brand, condition, images), browse and filter listings by category/location, send swap requests, negotiate through real-time chat, and complete swaps. An admin manages users/listings, monitors activity, and resolves disputes. The build requires 6–8 interconnected pages: Login, Clothing Listings, Item Detail, Swap Request, Chat, User Dashboard, and Admin Panel. Explicitly out of scope for Phase 1: online payments, AI-powered recommendations, AR virtual try-on, and a mobile app.

**Your task:**
1. Confirm you've read all five docs and summarize the current state from memory.md before starting.
2. Scaffold the project per architecture.md's folder structure.
3. Build strictly phase-by-phase per phases.doc.md, only starting a new phase once the prior phase's exit criteria are met.
4. Follow every rule in rules.md (tech choices, error-handling format, naming conventions, testing requirements) without exception.
5. Style every screen per design.md — mobile-responsive, using the defined color tokens and type scale.
6. Use realistic clothing data (real-sounding brands, sizes, conditions) — never placeholder/lorem-ipsum content in the final build.
7. After each feature or decision, append an entry to memory.md's "What Happened" log and update its "Currently Working" section — especially once the swap-value calculation formula is finalized.
8. If a requirement is ambiguous, stop and ask rather than guessing.
9. Deploy early to a live environment — a live deployed link is a mandatory final deliverable.

Start with **Phase 1: Login & Authentication** as defined in phases.doc.md.

## How to Use This

- Place rules.md, architecture.md, phases.doc.md, design.md, memory.md in a /docs folder in your repo
- Paste the Master Build Prompt above as your first message to your AI coding assistant
- At the start of every new session, have the assistant re-read memory.md first so it resumes exactly where it left off

- Move to the next phase only once the current phase's exit criteria (defined in phases.doc.md) are met
- Deploy to a live URL before final submission — this is a hard requirement per the PRD